

Package Management

Package repositories

The MSYS2 software distribution uses a port of `pacman` (known from Arch Linux) to manage (install, remove and update) binary packages and also to build those packages in the first place.

Packages in MSYS2 work like packages in popular Linux distributions. A package is an archive containing a piece of software. This normally means executable files, runtime libraries, data, shared and static link libraries, header files, config files, and manual pages. Packages also contain metadata, such as the software's name, description of its purpose, version number, vendor, checksum, and a list of dependencies necessary for the software to run properly. Upon installation, the files contained are extracted into your MSYS2 installation directory and the metadata are stored in a local database.

There are 5 package repositories, the "classical" ones **msys2**, **mingw32**, and **mingw64** and the newer **ucrt64**, and **clang64**. The packages in **msys2** are named just like on a Linux distribution, the packages in the others are prefixed by either `mingw-w64-i686-` for 32-bit packages, or `mingw-w64-x86_64-` for 64-bit packages with a secondary prefix `clang` or `ucrt` where applicable.

For more details about those see '[Environments](#)' and '[Package Naming](#)'.

Finding a package

If you want to find a specific package in the repository (and that package can or cannot be installed on your machine) you can use the following command:

```
pacman -Ss <name or part of the name of the package>
```

Example:

```
$ pacman -Ss openjp
```

```
mingw32/mingw-w64-i686-openjpeg 1.5.2-7
  An open source JPEG 2000 codec (mingw-w64)
mingw32/mingw-w64-i686-openjpeg2 2.1.0-7
  An open source JPEG 2000 codec (mingw-w64)
mingw64/mingw-w64-x86_64-openjpeg 1.5.2-7
  An open source JPEG 2000 codec (mingw-w64)
```



```
mingw64/mingw-w64-x86_64-openjpeg2 2.1.0-7 [installed]
  An open source JPEG 2000 codec (mingw-w64)
```

As you can see the `mingw-w64-x86_64-openjpeg2` package is installed, while the `mingw-w64-x86_64-openjpeg` package is **not** installed.

If you would like to search **only** among the packages which has been already installed, use the following command:

```
pacman -Qs <name or part of the name of the package>
```

Installing a package

If you want to install a package, use the following command:

```
pacman -S <name of the package>
```

If the package has dependencies which are not installed, `pacman` will ask you whether you would like to install the dependencies in the first place.

`pacman -S` also accepts virtual package names and package group names. Virtual package names can be often encountered with packages built from Git, e.g. `msys2-launcher-git` can be installed by requesting `msys-launcher`. Package groups simplify installation of related packages, e.g. install `base-devel` to get basic development tools. Please note that neither of those are real packages, so the commands below won't accept these names and you need to use the real package names instead.

Uninstalling a package

The following command will remove a package (but not its dependencies nor any files produced by running it):

```
pacman -R <name of the package>
```

Installing a specific version of a package or a stand-alone packages

Older (or pre-release) versions of packages can be installed directly from the package archive (`.tar.zst` or `.tar.xz`). [The data store](#) for the repositories contains older versions of packages, but beware that you might need to recursively find correct versions of dependencies for the desired package. Once downloaded, the package can be installed like this:

```
pacman -U <packagefile.tar.zst>
```

or

```
pacman -U <packagefile.tar.xz>
```

Finding dependencies of a package

You can use `pactree` to figure out which packages are needed to make a package working properly:

```
$ pactree mingw-w64-x86_64-gettext
```

```
mingw-w64-x86_64-gettext
├─mingw-w64-x86_64-expat
├─mingw-w64-x86_64-gcc-libs
│   ├─mingw-w64-x86_64-gmp
│   ├─mingw-w64-x86_64-libwinpthread-git provides mingw-w64-x86_64-libwinpthread
│   └─mingw-w64-x86_64-gcc-libgfortran
│       └─mingw-w64-x86_64-gcc-libs
└─mingw-w64-x86_64-libiconv
```

Alternatively you can use `pacman -Qi` to get the list of **direct** dependencies of a package:

```
$ pacman -Qi mingw-w64-x86_64-gettext
```

```
Name           : mingw-w64-x86_64-gettext
Version        : 0.19.7-1
Description     : GNU internationalization library (mingw-w64)
[...]
Depends On     : mingw-w64-x86_64-expat  mingw-w64-x86_64-gcc-libs
                  mingw-w64-x86_64-libiconv
```

Finding out which package a file belongs to

Use the following command to trace a file back to its owning package:

```
pacman -Qo <full file path>
```

Note that this operation only compares the file paths, so proper capitalization and the `.exe` suffix (if applicable) is required. Also note that this works only on installed packages, it will not scan the whole package repositories.

Finding which package will install the file you need

The two recommended tools that can scan a repository and find packages that contain specific files are `pacman -F` and `pkgfile`. Below are examples of `pacman -F` usage:

Call `pacman -Fy` to update your package database. To find an exact match, call `pacman -F <filename>` (don't include the path in the filename). To find a substring match, call `pacman -Fx <filename>`.

Note that this operation only compares the file paths, so proper capitalization and the `.exe` suffix (if applicable) is required.

Resources

- [Pacman on ArchWiki](#)
- [Pacman tips on ArchWiki](#)